

Composition Coherence Across Orchestration Layers: Why Naming the Three Layer Distinctions Matters in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 5, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. It draws on the source paper's §4.3 (the four-layer landscape), §13.3 (composition primitives as future work), §8.1 (the Stevens complementarity claim), and §11.4 (substrate as source of truth in composition); the composition pattern at the integrating level is in the source paper directly, and this note specializes it for defensive-publication purposes. It is the fifth and final note in the orchestration-layer-distinctions decomposition (A2.86–A2.90) of the foundational note A1.15.

Abstract

The CKS layer-distinction commitment formalized at A1.15 — that CKS is not a workflow engine, an agent framework, or a control plane — is not solely a separation claim. It is the architectural precondition for *coherent composition* between CKS and one or more orchestration-layer objects in hybrid deployments. The boundary-naming the three layer-distinction specializations provide (A2.87, A2.88, A2.89) is what makes the explicit naming of layers possible; without it, each composition pattern collapses into a specific incoherence failure mode in which one layer absorbs another's commitments. This note formalizes composition coherence across orchestration layers as a standalone architectural commitment. It states the four operational components — each layer responsible for state at its own timescale, substrate as the only layer persisting across all other layers' lifecycles, no inter-layer absorption of commitments, and the three operationally common composition patterns working coherently — names the specific incoherence failure mode for each composition pattern, distinguishes composition coherence from four adjacent patterns commonly conflated with it, names the failure modes that violate it, and provides an operational test.

1. Why composition coherence needs to be formalized as standalone

A1.15 commits to the layer distinctions; A2.86 named the composition pattern at the integrating level; A2.87, A2.88, and A2.89 specified the three layer-distinction specializations — CKS vs. workflow engine, CKS vs. agent framework, CKS vs. control plane. This note formalizes why those boundaries operationally matter: they are the precondition for hybrid systems in which CKS coordinates with one

or more orchestration-layer objects without confusion about which layer is doing what.

The motivating cases are deployments where CKS composes with one or more orchestration-layer objects. A CKS deployment may have a BPMN-based engine triggering cells that LangChain-style agents execute, hosted under enterprise IAM; may pair Stevens-style approval gates over operational access with substrate-level governance over content authority; or may combine all three orchestration-layer object families with the substrate as the unifying coordination-knowledge layer. Each scenario requires composition coherence — the layers must be named explicitly so that each operates architecturally as designed. As prior art, the commitment also forecloses architectures that present hybrids as unified single-layer systems: patentable derivations focused on hybrid AI architectures combining substrate, workflow, agent-framework, or control-plane layers are substantially more defensibly contested when the composition coherence commitment is publicly formalized as standalone.

The note also connects to the broader composition framework. A1.13 specifies the five composition requirements that any composition must satisfy (per A2.76–A2.80); A1.16 specifies the three composition patterns by which compositions structure (input to a cell, derivative view of substrate, separate concern). Composition coherence is what makes A1.13 and A1.16 operationally applicable to compositions involving orchestration-layer objects. The note is the structural parallel of A2.85, the closing-specialization note for A1.14 (the three adjacencies): both formalize why the boundaries the prior specializations provide enable coherent composition, but they operate on different sets of boundaries — A2.85 on memory-location adjacencies, this note on orchestration-layer distinctions.

2. The composition pattern, defined precisely

In the CKS pattern, a composition between a CKS deployment and one or more orchestration-layer objects exhibits **composition coherence** if and only if the following four operational components hold at all times during the composition's existence.

(a) Each layer responsible for state at its own timescale. Workflow engines are responsible for execution state for the duration of a process instance. Agent frameworks are responsible for execution state for the duration of an agent run. Control planes are responsible for operational state for the duration of an operational concern. The substrate is responsible for coordination state — what was decided, by whom, under what authority, with what rationale, against what conflicts — across cell runs and across the lifecycles of the other layers' objects. Each layer's state-responsibility is at its own architectural timescale.

(b) Substrate as the only layer persisting across all other layers' lifecycles. The substrate's persistence is by architectural commitment per A2.59 (Guarantee C: substrate changes are addressable across sessions), not by configurational convenience. The substrate survives every workflow process instance's termination, every agent run's termination, and every operational concern's resolution. The persistence is not contingent on retention policies, archival conventions, or operational settings in the other layers; it is a property of the substrate's architectural location.

(c) No inter-layer absorption of commitments. The substrate does not absorb the orchestration-layer objects' commitments — it does not become the workflow execution log, the agent framework's memory store, or the control plane's audit log. The orchestration-layer objects do not absorb the substrate's commitments — the workflow engine does not become the source of truth for what was decided; the agent framework's memory does not become the system's substrate; the control plane's access decisions do not become CKS governance decisions. Each layer operates within its own architectural commitments.

(d) Three operationally common composition patterns work coherently. The three patterns named in the foundational treatment — workflow-engine-triggers-CKS-cell, agent-framework-executes-CKS-cell, control-plane-authorizes-substrate-access — operate as architecturally distinct compositions that may coexist in a single deployment. A complete deployment may use all three patterns simultaneously without the layers collapsing into one another.

The four components together define composition coherence architecturally. Component (b) names the substrate's distinguishing commitment across all three composition patterns: the substrate is the only layer whose state persists by architectural commitment across the lifecycles of the other layers' objects. This persistence is what makes the substrate the appropriate location for coordination knowledge that must outlast every workflow process instance, every agent run, and every operational concern.

3. The three composition patterns and their specific incoherence failure modes

Each composition pattern has its own architectural shape, its own correspondence to A1.16's patterns, and its own specific incoherence failure mode if the layers are not named explicitly.

Workflow-engine-triggers-CKS-cell. The engine decides when the cell should run, given the workflow's process state and triggering conditions; the cell reads from and writes to the substrate under human-authored orchestration rules per A2.04. Two records exist after the cell runs: the engine records that it ran the cell; the substrate records what the cell decided, under what authority, with what rationale, against what conflicts. Neither substitutes for the other. The architectural pattern instantiates A1.16's Pattern A (input to a cell). The specific incoherence failure mode if the layers are not named is that the workflow engine is mistaken for the substrate — workflow execution state is read as coordination-knowledge state, and *what ran* is treated as *what was decided*. The boundary per A2.87 is what enables explicit naming.

Agent-framework-executes-CKS-cell. The framework manages the LLM execution machinery — prompting, tool calling, multi-step reasoning, multi-agent communication where applicable — with the substrate-mediator commitment of A1.04 holding throughout. Framework memory holds execution state for the duration of the cell's run; the substrate holds coordination state across cell runs. When the framework's agents terminate, framework memory is discarded or archived at the framework's layer; the substrate persists at its own layer, governed by the rights named in the human-governed commitment per A1.01. The specific incoherence failure mode if the layers are not named is that the agent framework's memory is mistaken for the substrate — framework memory is treated as coordination state, and the substrate's lifecycle-independent persistence per A2.59 is lost. The boundary

per A2.88 is what enables explicit naming.

Control-plane-authorizes-substrate-access. The control plane authenticates the user and authorizes the host-level access being exercised; the substrate enforces CKS-level governance authority once that access is granted. Control-plane access is necessary for CKS governance to be exercisable; CKS governance is the layer at which the substrate's content-level authority decisions are made. The architectural pattern instantiates A1.16's Pattern A or Pattern C depending on architecture — the control plane may operate as gating input to substrate access (Pattern A) or as a separate concern operating alongside the substrate (Pattern C). The specific incoherence failure mode if the layers are not named is that control-plane access is mistaken for CKS governance — host-level access is treated as substrate-level authority, and a user with infrastructure access is treated as a CKS governor. The boundary per A2.89 is what enables explicit naming.

A complete deployment may use all three patterns simultaneously: a workflow engine triggers CKS cells that an agent framework executes, hosted under a control plane that authorizes access. The three patterns compose with each other; the substrate operates as the coordination-knowledge layer for the entire deployment regardless of how many orchestration-layer objects are involved.

4. What composition coherence is NOT

Naming what composition coherence *is* requires naming what it *is not*, both as scoping limitations and as adjacent patterns commonly conflated with it.

It does not claim that compositions are necessarily preferable to single-layer CKS deployments; specific deployments may use single-layer CKS without orchestration-layer integration depending on operational concerns. It does not specify implementation patterns for layer coordination — cell-triggering APIs, substrate-aware agent memory interfaces, control-plane policies that read substrate content are explicitly future work per source paper §13.3. It does not foreclose compositions beyond the three named (compositions combining CKS with multiple orchestration-layer objects are operationally feasible when the multiple boundaries are all named explicitly), nor does it require all compositions to use the same A1.16 pattern, nor does it foreclose architectural evolution within compositions (orchestration-layer objects may be added, removed, or modified per Requirement E per A2.80; the architectural commitment is to coherence at any moment during the composition's existence).

The remaining four distinctions are between composition coherence and adjacent patterns commonly conflated with it.

Not architectural unification across layers. Architectural unification is the incorrect framing that compositions produce a single unified architecture combining all the layers' features. Composition coherence is different — compositions compose layers with explicit boundaries; each layer retains its architectural commitments; no unified architecture emerges from the composition. Audiences who understand composition as unification miss the boundary-naming commitment that makes coherence possible.

Not vendor-bundled multi-layer integration. Vendor-bundled integration is the operational pattern of vendors offering pre-integrated systems combining multiple architectural objects. Composition coherence is at the architectural level (boundary-naming, commitment preservation), not at the vendor-packaging level. Vendor bundles may operationally support composition coherence if architectural boundaries are preserved, or may violate it if layers are conflated within the bundle.

Not feature-additive composition. Feature-additive composition is the framing that compositions combine the features of constituent layers ("the deployment has both substrate features and workflow features"). Composition coherence preserves architectural commitments (which are not feature lists); each layer's commitments remain architecturally distinct; the composition is not a feature super-set.

Not orchestration-only or substrate-only deployments. Compositions are by definition multi-layer; deployments that are only orchestration-layer (without substrate commitments) or only substrate-layer (without orchestration-layer coordination) are not compositions in the architectural sense. The composition pattern operates specifically on multi-layer compositions where CKS commits at the coordination-knowledge layer alongside orchestration-layer objects at their respective layers.

5. Why composition coherence is load-bearing

Composition coherence is load-bearing for several CKS commitments. The integrating orchestration-layer-distinctions specification per A1.15 and A2.86 is operationally consequential only if coherence holds; without it, the boundaries A2.87, A2.88, A2.89 formalize would be inert. The five composition requirements per A1.13 (per A2.76–A2.80) — per-substrate human governance preservation, conflict preservation across boundaries, addressable provenance across boundaries, AI-as-mediator at every layer, human-selective composition — operate at the boundaries the layer-distinction specializations formalize and depend on the boundaries being explicit. The hybrid systems composition framework per A1.16 specifies the three patterns; coherence is what makes the patterns operationally applicable to compositions with orchestration-layer objects.

The human-governed commitment per A1.01 depends on coherence to be preserved in the CKS layer despite the orchestration-layer objects' different authority structures; without coherence, A1.01 collapses into the orchestration layer's authority posture (e.g., into control-plane access decisions). The AI-as-substrate-mediator commitment per A1.04 depends on coherence to be preserved in the CKS layer despite the orchestration layers' different AI-role assumptions; without coherence, A1.04 collapses into the orchestration layer's AI role (e.g., into the agent framework's autonomous-agent role). The temporal property per A2.07 (at-any-time governability) operates across orchestration-layer object lifecycles; coherence preserves the temporal commitment across compositions. The substrate-as-source-of-truth per A2.59 makes substrate changes addressable across sessions; coherence preserves the addressability across orchestration-layer object lifecycles. The Stevens complementarity claim per §8.1 depends on coherence — without it, complementarity collapses into substitution.

6. Failure modes that violate composition coherence

Each failure mode below names a way an implementation can fail by collapsing layers or absorbing commitments. *Workflow execution state read as coordination state*: the workflow engine is positioned

as the source of *what was decided and why*; component (c) of §2 fails for the workflow-engine layer. *Framework memory treated as substrate*: the agent framework's memory is treated as substrate-equivalent; component (b) of §2 fails because framework memory is bound to agent lifecycle. *Control-plane access read as CKS governance*: control-plane access is treated as conferring CKS governance authority; component (c) of §2 fails for the control-plane layer. *Composition presented as unified architecture*: the implementation presents the composition as a single architectural object rather than as composed layers with explicit boundaries; component (d) of §2 fails because audiences cannot tell which layer is doing what. *Boundaries documented but not operationally observable*: architectural documentation describes layer boundaries, but operational behavior conflates layers despite the documentation. *Substrate absorbs orchestration-layer commitments*: the substrate is asked to satisfy commitments belonging to orchestration layers — becoming the workflow execution log, carrying agent-execution state, or carrying operational audit logs. *Orchestration layers absorb substrate commitments*: orchestration layers are positioned as satisfying CKS's commitments — the workflow engine claiming to be the source of truth for decisions; the agent framework's memory claiming to be substrate; the control plane claiming to be CKS governance. *Substrate's lifecycle bound to orchestration-layer object lifecycle*: substrate persistence is bound to a workflow process instance, an agent run, or an operational concern; component (a) of §2 fails because the substrate's timescale is collapsed into another layer's timescale. *Composition-as-substitution rather than composition-as-coexistence*: compositions are treated as layer substitution; per §8.1, the layers are complementary, and substitution collapses the architectural distinction. *Implicit composition without explicit naming*: the architectural composition exists operationally but is not named in architectural documentation, deployment design, or system-level reasoning.

7. Operational test

A composition involving CKS and one or more orchestration-layer objects satisfies composition coherence if and only if all of the following are true at all times during the composition's existence:

1. Each layer is responsible for state at its own timescale per §2(a) — workflow engines for process-instance duration, agent frameworks for agent-run duration, control planes for operational-concern duration, substrate for coordination state across all other lifecycles.
2. The substrate persists across all other layers' lifecycles per §2(b) — substrate persistence is architectural per A2.59, not configurational; substrate survives every workflow process instance's termination, every agent run's termination, and every operational concern's resolution.
3. No inter-layer absorption of commitments per §2(c) — substrate does not absorb workflow execution logs, framework memory, or operational audit logs; orchestration-layer objects do not absorb CKS governance authority, source-of-truth, or coordination-state commitments.
4. The three operationally common composition patterns work coherently per §2(d) — workflow-engine-triggers-CKS-cell preserves both layers; agent-framework-executes-CKS-cell preserves both layers; control-plane-authorizes-substrate-access preserves both layers; a deployment using multiple patterns preserves all layers' commitments.

5. Composition requirements per A1.13 (per A2.76–A2.80) hold at the boundaries — per-substrate human governance is preserved, conflicts are preserved across boundaries, provenance is addressable across boundaries, AI-as-mediator commitment holds at every layer, and the composition decision is human-selective.

6. The substrate is operationally distinguishable from each orchestration-layer object per A2.87, A2.88, A2.89 — substrate is not workflow execution state, not agent-framework memory, not control-plane access decisions.

A composition that fails any of (1)–(6) does not satisfy composition coherence in the architectural sense, even if it operationally appears to combine the layers' features.

One-sentence test. If a composition's coordination-knowledge state — what was decided, by whom, under what authority, with what rationale — is held in a substrate that persists across all the orchestration-layer objects' lifecycles by architectural commitment, with each orchestration-layer object responsible for state at its own timescale and with the substrate's commitments not absorbed by any other layer, the composition is coherent; if any orchestration-layer object's state is being treated as substrate-equivalent (or vice versa), the composition is incoherent and the substrate's commitments are violated.

8. Why naming composition coherence as standalone matters

Implementations under pressure to deliver "comprehensive AI coordination platforms" consistently drift toward presentations that frame compositions as unified architectures or vendor-bundled integrations. The drift is steady because unified-architecture framings are commercially attractive, operationally simpler in the short term, and rhetorically appealing. Implementations that drift away from composition coherence produce systems where compositions collapse into one of their constituent layers operationally even when both layers are physically present. The downstream consequences manifest as commitment-absorption failures (A1.01, A1.04, and A2.07 commitments collapse into the orchestration layer's posture), prior-art failures (the architectural distinction is lost, and the composition is treated as covered by the orchestration-layer object's prior art), composition-requirement failures (A1.13's five requirements per A2.76–A2.80 cannot be operationally enforced because the boundaries are not observable), Stevens-complementarity failures (per §8.1, the layers are complementary; coherence preserves the complementarity), and architectural-credibility failures.

Naming composition coherence as a standalone architectural commitment gives downstream readers a precise specification of why the boundary-naming the prior specializations provide matters operationally. With this note complete and its companions A2.86–A2.89 already drafted, the orchestration-layer-distinctions decomposition is fully formalized: the integrating frame established the three-layer architectural model; the three layer-distinction specializations specified the boundaries between CKS and workflow engines, agent frameworks, and control planes; this composition-coherence specification specifies why the boundaries operationally matter — they enable coherent compositions in which CKS coordinates with orchestration-layer objects without confusion. Together, the five notes constitute the operational decomposition of A1.15.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Composition Coherence Across Orchestration Layers: Why Naming the Three Layer Distinctions Matters in the Coordination Knowledge Substrate Pattern*. May 5, 2026. ORCID: 0009-0004-8065-3235.